实验一：

```
`// `绘制椭圆的中点Bresenham算法扫描转换`
`void onMidPointEllispe(int a, int b) {`
    `int x, y;`
    float d1, d2;`

glBegin(GL_POINTS);
x = 0; y = b;
d1 = b * b + a * a * (-b + 0.25);
glVertex2i(x, y);
glVertex2i(-x, -y);
glVertex2i(-x, y);
glVertex2i(x, -y);

// 第一个 while 循环用于绘制椭圆的上半部分
while (b * b * (x + 1) < a * a * (y - 0.5)) {
    if (d1 <= 0) {
        d1 += b * b * (2 * x + 3);
        x++;
    } else {
        d1 += b * b * (2 * x + 3) + a * a * (-2 * y + 2);
        x++; y--;
    }
    glVertex2f(x, y);
    glVertex2f(-x, -y);
    glVertex2f(-x, y);
    glVertex2f(x, -y);
}

// 第二个 while 循环用于绘制椭圆的下半部分
d2 = b * b * (x + 0.5) * (x + 0.5) + a * a * (y - 1) * (y - 1) - a * a * b * b;
while (y > 0) {
    if (d2 <= 0) {
        d2 += b * b * (2 * x + 2) + a * a * (-2 * y + 3);
        x++; y--;
    } else {
        d2 += a * a * (-2 * y + 3);
        y--;
    }
    glVertex2f(x, y);
    glVertex2f(-x, -y);
    glVertex2f(-x, y);
    glVertex2f(x, -y);
}

glEnd();
glFlush();
// 计算机图形学computer Graphics
// 仔细查看上述代码，回答下述问题：
```

```
// 1. 变量 x, y, d1, 和 d2 的作用分别是什么？
// x 和 y 是当前绘制的点的坐标，d1 和 d2 是中点Bresenham算法中用于决策下一步坐标变化的判定条件。

// 2. 在代码的第一个 while 循环中，条件 b * b * (x + 1) < a * a * (y - 0.5) 是如何选择的？
// 条件 b * b * (x + 1) < a * a * (y - 0.5) 是为了确保在椭圆的上半部分绘制，具体条件是基于中点Bresenham算法的性质选择的。

// 3. 在第一个 while 循环中，当 d1 大于 0 时，为什么 x++, y--？请说明计算的逻辑。
// 当 d1 大于 0 时，表示下一个点在当前点的右上方，因此 x 增加，y 减少。

// 4. 第二个 while 循环中，为什么选择条件 y > 0 作为循环结束条件？
// 第二个 while 循环是绘制椭圆的下半部分，因此当 y 到达椭圆的底部时结束。

// 5. 解释 d2 的计算逻辑，并说明为什么它用于第二个 while 循环。
// d2 是中点Bresenham算法的决策参数，用于判断下一个点的位置。它包含了椭圆方程的左侧减去右侧的部分，当 d2 小于等于 0 时，下一个点在当前点的右上方，否则在右方。

// 6. 函数中的 glVertex2f 是用于绘制椭圆上的点，你认为可以通过修改代码实现以不同颜色绘制椭圆，该如何修改？
// 可以通过在 glBegin(GL_POINTS) 之前使用 glColor3f(r, g, b) 来设置绘制点的颜色。

// 7. 如何修改代码以将椭圆绘制在屏幕上指定的区域，而不是默认的位置？
// 可以在 glVertex2i(x, y) 和 glVertex2i(-x, -y) 之前加上偏移值，即 x = x + offsetX, y = y + offsetY。

// 8. 如果需要绘制更大的椭圆，你会如何修改代码以适应更大的屏幕空间？
// 可以通过调整参数 a 和 b 的值来适应更大的屏幕空间。
```

}

实验二：

```
// 裁剪直线的算法
int Clip_Top(float p, float q, float &umax, float &umin) {
    float r = 0.0;
    if (p < 0.0) {
        // 线段从裁剪窗口外部延伸到内部，取最大值r并更新umax
        r = q / p;
        if (r > umin) return 0;  // umax>umin的情况，弃之
        else if (r > umax)  umax = r;
    } else if (p > 0.0) {
        // 线段从裁剪窗口内部延伸到外部,取最小值r并更新umin
        r = q / p;
        if (r < umax) return 0;  // umax>umin的情况，弃之
        else if (r < umin)  umin = r;
    } else {
        // p=0时，线段平行于裁剪窗口
        if (q < 0.0) return 0;
    }
    return 1;
}
```

```
// 直线裁剪函数
void Line_Clipping(vector<point> &points, rect &winRect) {
    point &p1 = points[0], &p2 = points[1];
    float dx = p2.x - p1.x, dy = p2.y - p1.y, umax = 0.0, umin = 1.0;
    point p = p1;
if (Clip_Top(-dx, p1.x - winRect.w_xmin, umax, umin))  // 左边界
    if (Clip_Top(dx, winRect.w_xmax - p1.x, umax, umin))  // 右边界
        if (Clip_Top(-dy, p1.y - winRect.w_ymin, umax, umin))  // 下边界
            if (Clip_Top(dy, winRect.w_ymax - p1.y, umax, umin))  // 上边界
            {
                // 求裁剪后直线新端点
                p1.x = (int)(p.x + umax * dx);
                p1.y = (int)(p.y + umax * dy);
                p2.x = (int)(p.x + umin * dx);
                p2.y = (int)(p.y + umin * dy);
            }
```

}

仔细查看上述代码，回答下述问题：

1. 以下是对提供的问题的详细解答：
    1. **`Clip_Top` 函数中的参数 `p` 和 `q` 分别表示什么意义？**
        - `p` 表示直线的斜率，而 `q` 表示直线方程中的常数项。这两个参数用于判断直线与裁剪窗口的交点情况。
    2. **在 `Clip_Top` 函数中，为什么要计算和更新 `umax` 和 `umin`？**
        - `umax` 和 `umin` 用于记录直线与裁剪窗口边界的交点的参数值。通过比较和更新这两个值，可以确保找到直线在裁剪窗口内的有效部分。
    3. **为什么在 `Line_Clipping` 函数中要比较左、右边界，然后根据裁剪窗口的左、右边界，求 `umax`？**
        - 比较左、右边界并求 `umax` 的目的是确保直线在裁剪窗口的水平范围内。这一步是为了找到直线与裁剪窗口左右边界的交点，并更新 `umax`。
    4. **为什么在 `Line_Clipping` 函数中要比较下、上边界，然后根据裁剪窗口的下、上边界，求 `umin`？**
        - 比较下、上边界并求 `umin` 的目的是确保直线在裁剪窗口的垂直范围内。这一步是为了找到直线与裁剪窗口上下边界的交点，并更新 `umin`。
    5. **如果 `umax > umin`，裁剪窗口和直线无交点，这个条件的判断是如何实现的？**
        - 在 `Clip_Top` 函数中，如果 `umax > umin`，则返回 0，表示裁剪窗口和直线无交点。这是因为 `umax` 表示直线与裁剪窗口右边界的交点，`umin` 表示直线与裁剪窗口左边界的交点。如果右边界的交点比左边界的交点更靠左，说明直线在裁剪窗口范围外。
    6. **在 `Clip_Top` 函数中，当 `p` 和 `q` 的值为什么会影响到 `r` 的计算和更新？**
        - `r` 的计算是通过 `r = q / p` 进行的。`p` 表示直线的斜率，`q` 表示直线方程中的常数项。`r` 的值用于确定直线与裁剪窗口的交点位置。当 `p` 和 `q` 的值变化时，`r` 的计算结果也会相应变化，影响到最终的交点位置。

7. **为什么在 `Clip_Top` 函数的最后返回 1 或 0？**
   - 在函数中，当 `umax > umin` 时，表示直线与裁剪窗口无交点，返回 0；否则返回 1，表示有交点。这样的设计方便在直线裁剪函数中判断是否需要进行后续的计算和更新。
8. **`Clip_Top` 函数和 `Line_Clipping` 函数中的 `glBegin(GL_POINTS)` 和 `glFlush()` 语句有什么作用？是否可以删除？为什么？**
   - 这两个语句用于在图形界面中绘制点，并刷新图形缓冲区。在这两个函数中，这些语句可能是错误的，因为它们似乎不是直线裁剪算法的一部分。这可能是来自其他与图形绘制相关的代码片段。
9. **如果需要修改代码以支持更多的裁剪窗口形状，你认为需要怎样的改动？**
   - 如果需要支持更多的裁剪窗口形状，可能需要修改裁剪算法的实现以适应不同形状的裁剪窗口。这可能包括更改边界比较和交点计算的逻辑，以满足新的裁剪窗口形状。
10. **在 `Line_Clipping` 函数中，`int` 类型的坐标值是否能够精确表示直线的端点？如果不能，你会采取什么措施来解决这个问题？**
   - `int` 类型的坐标值可能无法精确表示直线的端点，特别是在进行计算和裁剪时可能会引入舍入误差。为了解决这个问题，可以考虑使用浮点数类型或者其他更高精度的数据类型来表示直线的端点坐标，以提高精度并减少计算误差。

希望这些解答对于理解直线裁剪算法和相关概念有所帮助。如果还有其他问题或需要进一步解释，请随时告诉我。

```
//lineClipping.h 头文件中的代码
#include <iostream>
#include <vector>
#include <windows.h>
#include <GL/glut.h>
using namespace std;

//点类型point
typedef struct Point {
    int x, y;
    Point(int a = 0, int b = 0)
    {
        x = a, y = b;
    }
} point;

//矩形类型rect
typedef struct Rectangle {
    float w_xmin, w_ymin;
    float w_xmax, w_ymax;
    Rectangle(float xmin = 0.0, float ymin = 0.0, float xmax = 0.0, float ymax =
0.0) {
        w_xmin = xmin; w_ymin = ymin;
        w_xmax = xmax; w_ymax = ymax;
```

```
    }
} rect;

// 线段裁剪函数声明
int Clip_Top(float p, float q, float &umax, float &umin);

// 直线裁剪函数声明
void Line_Clipping(vector<point> &points, rect &winRect);
```

以下是对提供的问题的详细解答：

1. **`point` 和 `rect` 结构体分别表示什么？**
   - `point` 结构体表示一个二维平面上的点，包含 `x` 和 `y` 两个整型成员变量，用于表示点的坐标。
   - `rect` 结构体表示一个矩形区域，包含 `w_xmin`、`w_ymin`、`w_xmax` 和 `w_ymax` 四个浮点型成员变量，分别表示矩形的左下角和右上角坐标。
2. **`Clip_Top` 函数的输入参数 `p` 和 `q` 分别表示什么？**
   - `p` 表示直线方程中的一个参数，`q` 表示直线方程中的另一个参数。这两个参数用于描述直线的数学特性，参与直线与裁剪窗口的交点计算。
3. **在 `Rectangle` 结构体中，`float` 类型的成员变量与直线裁剪算法有何关联？**
   - `float` 类型的成员变量用于表示矩形的坐标值，因为直线裁剪算法通常涉及到浮点数计算，以更精确地表示直线与矩形边界的交点。
4. **为什么 `Clip_Top` 函数的参数中有引用类型的参数 `&umax` 和 `&umin`？**
   - 引用类型的参数 `&umax` 和 `&umin` 用于传递参数并修改其值，以便在函数外部获取到 `umax` 和 `umin` 的计算结果。
5. **头文件中包含的 `<windows.h>` 和 `<GL/glut.h>` 是用于什么目的？**
   - `<windows.h>` 头文件通常用于在 Windows 操作系统下进行图形界面的编程。而 `<GL/glut.h>` 则是 OpenGL Utility Toolkit（GLUT）库的头文件，用于处理窗口管理、输入事件等图形界面相关的功能。
6. **在 `Line_Clipping` 函数中，`vector<point> &points` 和 `rect &winRect` 的作用是什么？**
   - `vector<point> &points` 用于表示直线的两个端点，`rect &winRect` 表示裁剪窗口的信息。这两个参数是函数的输入，用于描述直线和裁剪窗口的位置。
7. **如果要在头文件中添加新的函数声明，应该注意什么？**
   - 在头文件中添加新的函数声明时，应注意避免重复声明，可以使用预处理器指令（`#ifndef`，`#define`，`#endif`）防止头文件的重复包含。还应确保函数声明的格式正确，包括函数名、参数列表、返回类型等。
8. **为什么使用 `typedef` 关键字定义了 `point` 和 `rect` 结构体的别名？**
   - 使用 `typedef` 关键字定义别名有助于提高代码的可读性，使得代码中可以使用简洁的名称 `point` 和 `rect` 来代替 `struct Point` 和 `struct Rectangle`。
9. **在 `Line_Clipping` 函数中，`glut` 库的函数 `glBegin(GL_POINTS)` 和 `glFlush()` 是否有必要？为什么？**

- 在 `Line_Clipping` 函数中，这两个 `glut` 库的函数可能是错误的，因为该函数似乎是直线裁剪算法，与图形绘制无关。这可能是来自其他与图形绘制相关的代码片段。在直线裁剪函数中，这两个函数调用可能不必要。

10. **为什么在 `Rectangle` 结构体中使用了 `float` 类型而不是 `int` 类型？这对直线裁剪算法有何影响？**
    - 使用 `float` 类型而不是 `int` 类型可以提高对于浮点数计算的精度，特别是在直线裁剪算法中，可能涉及到浮点数的计算。这有助于更精确地表示直线与矩形边界的交点，提高算法的准确性。

实验三

```
//给定的代码段
#include <gl/glut.h>
#include <math.h>

//点类型point3D
typedef struct Point3D {
    double x, y, z;
    Point3D(double a = 0.0, double b = 0.0, double c = 0.0)
    {
        x = a, y = b; z = c;
    }
} point3D;

float PI = 3.14159f;

//根据菜单menuValue选择的三视图，建立视角视觉效果
void setLookAt(int menuValue, point3D eye, point3D at);

//根据菜单menuValue选择的三视图，根据右手原则，围绕某坐标轴旋转angle，建立视角视觉效果
void setLookAt(int menuValue, float angle, point3D eye, point3D at);

//控制照相机的三维旋转角度，将场景坐标系映射到eye的观察坐标系，建立第一人称视角视觉效果
void setLookAt(const GLfloat x_angle, const GLfloat y_angle, const GLfloat z_angle,
point3D eye, point3D at);
```

以下是对提供的问题的详细解答：

1. **`point3D` 结构体的成员变量 `x`、`y` 和 `z` 分别表示什么？**
    - `x`、`y` 和 `z` 分别表示三维空间中的一个点的坐标，其中 `x` 表示横坐标，`y` 表示纵坐标，`z` 表示垂直坐标。
2. **`PI` 的值是多少？为什么在代码中使用 `PI`？**
    - `PI` 的值为 3.14159。在代码中使用 `PI` 是为了表示圆周率，通常用于进行角度和弧度之间的转换以及三角函数的计算。
3. **`setLookAt` 函数的重载形式有几种？分别是针对什么情况的？**

- `setLookAt` 函数有三种重载形式，分别是：
  1. 正视图、俯视图、侧视图的设定，针对三个基本视图的情况。
  2. 根据菜单选择的三视图，绕某坐标轴旋转一定角度的设定，适用于用户选择不同视图并旋转的情况。
  3. 控制照相机的三维旋转角度，建立第一人称视角，适用于自由控制相机位置和方向的情况。

4. **在第一个 `setLookAt` 函数中，`gluLookAt` 函数的参数是什么？为什么有三个不同的视图？**
   - `gluLookAt` 函数的参数包括视点的位置（`eye` 参数）、观察点的位置（`at` 参数）和上方向（`up` 参数）。有三个不同的视图是因为函数根据用户选择的三视图（正视图XOZ、俯视图XOY、侧视图YOZ），分别设置了视点和观察点的位置，以及上方向。
5. **第二个 `setLookAt` 函数中，`Rot` 是如何计算的？为什么要进行正弦和余弦运算？**
   - `Rot` 的计算通过将用户输入的角度转换为弧度。正弦和余弦运算是因为在绕坐标轴旋转时，需要计算新的坐标轴方向。正弦和余弦值用于计算旋转后的坐标。
6. **在第三个 `setLookAt` 函数中，为什么要进行角度的正弦和余弦运算？**
   - 角度的正弦和余弦运算用于计算观察坐标系的新基向量，以便在场景坐标系中进行照相机的旋转。
7. **在第三个 `setLookAt` 函数中，`ux`、`uy`、`uz`、`vx`、`vy`、`vz`、`nx`、`ny`、`nz` 的计算分别代表什么？**
   - 这些变量分别代表观察坐标系的新基向量，其中 `(ux, uy, uz)` 是 `u` 轴，`(vx, vy, vz)` 是 `v` 轴，`(nx, ny, nz)` 是 `n` 轴。
8. **为什么要在第三个 `setLookAt` 函数中计算新的观察坐标系的基向量？**
   - 计算新的观察坐标系的基向量是为了根据用户设定的三维旋转角度，将场景坐标系映射到照相机的观察坐标系，从而实现相机的旋转。
9. **`camera` 结构体表示什么？在第三个 `setLookAt` 函数中如何使用 `camera` 结构体？**
   - `camera` 结构体表示照相机在观察坐标系中的位置。在第三个 `setLookAt` 函数中，通过计算新的观察坐标系的基向量，将场景坐标系中的 `eye` 映射到观察坐标系中的 `camera`。
10. **在 `setLookAt` 函数中，`gluLookAt` 函数是用来做什么的？**
    - `gluLookAt` 函数用于设置视图矩阵，定义观察坐标系，确定照相机的位置和观察点，以及指定上方向。这样可以控制场景在屏幕上的显示，实现不同视图和视角的效果。

```
#include <windows.h>
#include <gl/glut.h>
#include "setLookAt.h"
#include "getAxis.h"          // 给出轴长，定义三维坐标轴图段，x轴为红色、y为绿、z为蓝
#include "getTetrahedron.h" // 给出4个三维坐标点ABCD，定义四面体图段

using namespace std;

static GLsizei iMode = 1;     // 三视图菜单项
float axisLength = 20.0;      // 三维坐标轴长
GLuint objectID, axisID;      // 四面体对象ID、坐标轴ID
```

```
int winWidth = 400, winHeight = 200;  // 窗口的宽度和高度
static float angle = 45.0;   // 绕坐标轴的旋转角
static point3D s_eye(5.0, 5.0, 5.0);   // 观察点位置
static point3D s_at(0.0, 0.0, 0.0);    // 视点中心
static point3D A(2.0f, 0.0f, 0.0), B(2.0f, 1.0f, 0.0), C(0.0f, 1.0f, 0.0), D(1.0f,
1.0f, 1.0);
// 四面体四点坐标

void initial(void)
{
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); // 设置窗口背景颜色为白色
}

void ChangeSize(GLsizei w, GLsizei h)  // 改变窗口大小
{
    glPolygonMode(GL_FRONT_AND_BACK, GL_LINE); // 多边形模式为线框
    winWidth = w;
    winHeight = h;
    glViewport(0, 0, w, h);  // 指定窗口显示区域
    glMatrixMode(GL_PROJECTION);  // 设置投影参数
    glLoadIdentity();
}

void triangle(GLsizei mode)  // 左右视图的绘制，mode决定选左or选右
{
    glPolygonMode(GL_FRONT_AND_BACK, GL_LINE); // 多边形模式为线框
    GLint vp[4];
    glGetIntegerv(GL_VIEWPORT, vp);  // 获得视区的左下角坐标、宽度和高度信息
    int w = vp[2], h = vp[3];
    float aspect;
    if (w < h)
    {
        aspect = (float)w / (float)h;
    }
    else
    {
        aspect = (float)h / (float)w;
    }
    glMatrixMode(GL_MODELVIEW);  // 指定当前操作模型视图矩阵堆栈
    glLoadIdentity();            // 恢复初始坐标系，将当前点移动到屏幕中心

    if (mode == 1)  // 左视区
    {
        setLookAt(iMode, s_eye, s_at);  // 根据菜单iMode选择三视图，建立视角视觉效果
        glMatrixMode(GL_PROJECTION);    // 设置投影参数
        glLoadIdentity();
        // 定义左视区的三维正交观察可视体，创建一个正射投影矩阵
        glOrtho(-5.0, 5.0, -5.0, 5.0, 5.0, -5.0);
    }
    else  // 右视区
    {
        // 根据菜单iMode选择三视图，根据右手原则围绕某坐标轴旋转angle，建立视角视觉效果
```

```
        setLookAt(iMode, angle, s_eye, s_at);
        glMatrixMode(GL_PROJECTION);  // 设置投影参数
        glLoadIdentity();
        gluPerspective(45.0f, aspect, 0.0f, 10.0f);  // 定义右视区的三维透视观察可视体
    }
    glCallList(objectID);  // 显示列表所存储的函数
    glCallList(axisID);
}

void SpecialKeys(int key, int x, int y)  // 功能键上/下键控制旋转角度
{
    if (key == GLUT_KEY_UP)  // 上键递减
    {
        angle = angle - 1;
    }
    else if (key == GLUT_KEY_DOWN)  // 下键递增
    {
        angle = angle + 1;
    }
    glutPostRedisplay();  // 刷新
}

void ProcessMenu(int value)  // 处理菜单响应
{
    iMode = value;
    glutPostRedisplay();
}

void Display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 0.0, 0.0);
    axisID = getAxis(axisLength);                // 根据轴长axisLength定义XYZ三维坐标系
    objectID = getTetrahedron(A, B, C, D);  // 4点ABCD定义四面体图元
    glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);  // 多边形模式为线框
    // 以下为左视区
    glViewport(0, 0, 200, 200);  // 指定从0，0开始，长宽均为200的视区
    triangle(1);
    // 以下为右视区
    glViewport(200, 0, 200, 200);  // 指定从200，0开始，长宽均为200的视区
    triangle(2);
    glFlush();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowPosition(100, 100);
    glutInitWindowSize(winWidth, winHeight);
    glutCreateWindow("shanchuxiong");  // 多视区
    // 创建菜单并定义菜单回调函数
```

```
    glutCreateMenu(ProcessMenu);
    glutAddMenuEntry("To__XOZ(V)", 1);
    glutAddMenuEntry("To__XOY(H)", 2);
    glutAddMenuEntry("To__YOZ(W)", 3);
    glutAttachMenu(GLUT_RIGHT_BUTTON);  // 主菜单与鼠标右键关联
    initial();
    glutDisplayFunc(Display);        // 指定窗口重绘响应函数
    glutReshapeFunc(ChangeSize);     // 指定窗口大小改变响应函数
    glutSpecialFunc(SpecialKeys);    // 指定功能键响应函数
    glutMainLoop();
    return 0;
}
```

1. **`setLookAt` 函数有几种重载形式？每种形式分别用于什么情况？**
   - `setLookAt` 函数有三种重载形式：
     1. 第一种形式用于根据菜单项选择三视图，建立视角视觉效果。
     2. 第二种形式用于根据菜单项选择三视图，根据右手原则围绕某坐标轴旋转一定角度，建立视角视觉效果。
     3. 第三种形式用于控制照相机的三维旋转角度，将场景坐标系映射到照相机的观察坐标系，建立第一人称视角视觉效果。
2. **在 `triangle` 函数中，`glOrtho` 和 `gluPerspective` 分别用于什么目的？**
   - `glOrtho` 用于定义左视区的三维正交观察可视体，创建一个正射投影矩阵。`gluPerspective` 用于定义右视区的三维透视观察可视体。
3. **在 `Display` 函数中，为什么先绘制左视区再绘制右视区？**
   - 先绘制左视区再绘制右视区是为了确保在绘制右视区时视图矩阵不会受到左视区的影响。这样能够在每个视区中正确地呈现相应的图形。
4. **在 `Display` 函数中，`getAxis` 和 `getTetrahedron` 函数分别返回什么值？**
   - `getAxis` 函数返回一个代表坐标轴的显示列表的 ID，`getTetrahedron` 函数返回一个代表四面体的显示列表的 ID。
5. **在 `SpecialKeys` 函数中，为什么使用 `glutPostRedisplay` 函数？**
   - `glutPostRedisplay` 函数用于通知系统窗口需要进行重新绘制。在按下功能键后，通过调用该函数，程序可以在下一次主循环中触发 `Display` 函数，实现图形的更新。
6. **`axisID` 和 `objectID` 分别用于什么？**
   - `axisID` 存储坐标轴的显示列表 ID，`objectID` 存储四面体的显示列表 ID。这些 ID 在绘制时用于调用相应的显示列表。
7. **如何通过右键菜单选择不同的三视图？**
   - 通过右键点击窗口，弹出的菜单中有三个选项："To**XOZ(V)", "To**XOY(H)", "To__YOZ(W)"。用户可以选择不同的三视图，菜单项的选择由 `ProcessMenu` 函数处理。
8. **在 `main` 函数中，`glutCreateMenu` 函数的作用是什么？**
   - `glutCreateMenu` 函数用于创建一个弹出式菜单。在这个例子中，该菜单包含了三个菜单项，用于选择不同的三视图。

9. **在 `ChangeSize` 函数中，为什么设置 `glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)`？**
   - `glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)` 设置多边形模式为线框模式，使得绘制的图形以线框的形式呈现，而不是填充。这是因为三维坐标轴和四面体图元是以线框的形式绘制的。
10. **在 `setLookAt` 函数中，`glLoadIdentity` 的作用是什么？**
   - `glLoadIdentity` 用于将当前矩阵设置为单位矩阵，即恢复初始坐标系，清空之前的变换。

实验四

```
#include <vector>
using namespace std;
//根据点序列向量数组points，绘制折线
void drawPolygonalLine(vector<point> &points,color &c );
//根据点序列向量数组points，绘制折线
void drawPolygonalLine(vector<point> &points ,point &p,color &c);
//如果控制点数n<4，绘制一段n-1次Bezier曲线段，否则绘制一段三次Bezier曲线段
void drawBezierCurve(vector<Point> &points,color &c);
//通过（m×3+1）控制点，绘制m段n-1次ezier曲线段
void drawBezierCurve(vector<Point> &points,int n,color &c);
```

1. **在 `drawPolygonalLine` 函数中，参数 `color &c` 有什么作用？**

   答：`color &c` 参数用于指定绘制折线的颜色，其中 `c` 是一个包含红、绿、蓝三个分量的颜色结构体。

2. **`drawBezierCurve` 函数分为两个版本，它们的主要区别是什么？**

   答：`drawBezierCurve` 函数的两个版本区别在于参数列表，其中一个版本接受一个颜色参数，而另一个版本接受一个整数 `n`，代表贝塞尔曲线的次数。

3. **`drawBezierCurve` 函数中的 `n` 参数代表什么意义？**

   答：在 `drawBezierCurve` 函数中，`n` 参数代表贝塞尔曲线的次数，决定了曲线的阶数。通常，三次贝塞尔曲线（cubic Bezier curve）是最常见的。

4. **在 `drawBezierCurve` 函数中，为什么要判断 `n < 4`？**

   答：判断 `n < 4` 是因为对于三次贝塞尔曲线（cubic Bezier curve），至少需要四个控制点。如果 `n` 小于 4，说明控制点数量不足以生成有效的三次贝塞尔曲线，因此会绘制一段 `n-1` 次的 Bezier 曲线。

5. **在 `drawBezierCurve` 函数中，`(m×3+1)` 控制点的选择有什么特殊的含义？**

   答：这个选择表示一段贝塞尔曲线由多个三次贝塞尔曲线组成，其中每个三次贝塞尔曲线需要四个控制点（3×3+1 = 10 个点），加上一个额外的控制点作为下一段贝塞尔曲线的起点。这样的设计可以保证贝塞尔曲线的光滑连接。

6. **`drawPolygonalLine` 函数中的第二个版本，参数 `point &p` 有什么作用？**

   答：`point &p` 参数用于指定绘制折线的起始点，即折线的第一个顶点。

7. **在 `drawBezierCurve` 函数中，`points` 参数代表什么？**

   答：`points` 参数是一个存储控制点坐标的向量，用于定义贝塞尔曲线的形状。

8. **在 `drawPolygonalLine` 函数中，`points` 参数的数据结构是什么？**

答：`points` 是一个存储 `point` 结构体的向量，每个 `point` 结构体表示折线的一个顶点坐标。

9. **这段代码中涉及的绘图是二维还是三维？**

   答：这段代码涉及的是二维绘图，因为点的数据结构只包含 x 和 y 坐标。

10. **如果需要在绘制的时候改变线宽，应该在代码中的哪里进行设置？**

    答：在 OpenGL 中，可以通过 `glLineWidth` 函数在绘制前设置线宽，所以应该在绘制折线或曲线之前设置。

11. **在 `drawBezierCurve` 函数中，如果传入的 `n` 大于 `points` 中点的数量，会发生什么情况？**

    答：如果传入的 `n` 大于 `points` 中点的数量，可能会导致数组越界或者无效的计算，程序的行为可能是不可预测的。因此，调用者应确保传入的参数是有效的。

12. **`drawBezierCurve` 函数中的 `color &c` 参数是用于指定什么颜色的？**

    答：`color &c` 参数用于指定绘制贝塞尔曲线时的颜色。

13. **如果要在 `drawPolygonalLine` 函数中添加绘制线段的粗细，应该如何修改代码？**

    答：可以在绘制前使用 `glLineWidth` 函数设置线段的粗细，可以通过增加一个新的参数传递粗细值。

14. **在 `drawBezierCurve` 函数中，为什么需要绘制多段 Bezier 曲线？**

    答：绘制多段 Bezier 曲线可以创建更复杂的曲线形状，通过将多个小段的曲线连接起来，可以获得光滑的整体曲线。

15. **对于 `drawBezierCurve` 函数中的绘制算法，是否了解贝塞尔曲线的基本原理？**

    答：了解贝塞尔曲线的基本原理是理解这段代码的关键。贝塞尔曲线是通过多个控制点插值生成的光滑曲线，其形状由这些控制点的位置决定。

```
#include <vector>
using namespace std;
#include <GL/glut.h>
#include "graphicType.h"
//绘制折线
void drawPolygonalLine(vector<point> &points,color &c ){//根据点序列向量数组points，绘制
折线
    glColor3f(c.r, c.g, c.b);
    glBegin (GL_LINE_STRIP);
        for(int i=0;i<points.size();i++){
            glVertex2i(points[i].x, points[i].y);
        }
    glEnd();
}
//绘制折线
void drawPolygonalLine(vector<point> &points ,point &p,color &c){//根据点序列向量数组
points，绘制折线
    glColor3f(c.r, c.g, c.b);
    glBegin (GL_LINE_STRIP);
        for(int i=0;i<points.size();i++){
            glVertex2i(points[i].x, points[i].y);
        }
        glVertex2i(p.x, p.y);
```

```
    glEnd();
}
//如果控制点数n<4，绘制一段n-1次Bezier曲线段，否则绘制一段三次Bezier曲线
void drawBezierCurve(vector<Point> &points,color &c){
    GLfloat ControlP[4][3];
    int iPointNum=points.size();
    int i;
    if(iPointNum<4){//控制点数<4，绘制一段n-1次Bezier曲线段
        for(i=0;i<iPointNum;i++){
            ControlP[i][0]=points[i].x;
            ControlP[i][1]=points[i].y;
            ControlP[i][2]=0.0;
        }
    }
    else{
        for(i=0;i<4;i++){//控制点数>=4，绘制一段三次Bezier曲线段
            ControlP[i][0]=points[i].x;
            ControlP[i][1]=points[i].y;
            ControlP[i][2]=0.0;
        }
    }
    glColor3f(c.r, c.g, c.b);
    glPointSize(2);
    if(iPointNum>4)
        glMap1f(GL_MAP1_VERTEX_3,0.0,1.0,3,4,*ControlP);//定义一维取值器
    else
        glMap1f(GL_MAP1_VERTEX_3,0.0,1.0,3,iPointNum,*ControlP);//定义一维取值器
    glEnable(GL_MAP1_VERTEX_3);
    glMapGrid1f(100,0.0,1.0);//生成均匀分布的一维网格参数值
    glEvalMesh1(GL_LINE, 0, 100);//绘制Bezier曲线
}
//通过（m×3+1）控制点，绘制m段n-1次ezier曲线段
void drawBezierCurve(vector<Point> &points,int n,color &c){
}
```

1. **在 `drawPolygonalLine` 函数中，`color &c` 参数的作用是什么？**

   答：`color &c` 参数用于指定绘制折线的颜色，其中 `c` 是一个包含红、绿、蓝三个分量的颜色结构体。

2. **`drawBezierCurve` 函数的两个版本分别用于绘制什么类型的曲线？**

   答：`drawBezierCurve` 函数有两个版本，其中一个版本用于绘制一段 `n-1` 次的 Bezier 曲线段，另一个版本用于绘制一段三次 Bezier 曲线段。

3. **在 `drawBezierCurve` 函数中，为什么要判断 `iPointNum < 4`？**

   答：判断 `iPointNum < 4` 是因为对于三次贝塞尔曲线（cubic Bezier curve），至少需要四个控制点。如果 `iPointNum` 小于 4，说明控制点数量不足以生成有效的三次贝塞尔曲线，因此会绘制一段 `n-1` 次的 Bezier 曲线。

4. **在 `drawBezierCurve` 函数中，为什么要使用 `glMap1f`、`glEnable` 和 `glMapGrid1f` 函数？**

答：这些函数用于配置和启用 OpenGL 的一维 evaluators，以便绘制 Bezier 曲线。`glMap1f` 用于定义一维取值器，`glEnable(GL_MAP1_VERTEX_3)` 启用该取值器，`glMapGrid1f` 生成均匀分布的一维网格参数值，而 `glEvalMesh1` 用于实际绘制 Bezier 曲线。

5. **在 `drawBezierCurve` 函数中，`glEvalMesh1(GL_LINE, 0, 100)` 的参数含义是什么？**

   答：`glEvalMesh1(GL_LINE, 0, 100)` 用于绘制 Bezier 曲线，其中 `GL_LINE` 表示使用线段进行绘制，而 `0` 和 `100` 分别表示在参数范围 `[0.0, 1.0]` 内生成的网格数。

6. **在 `drawBezierCurve` 函数的第二个版本（参数包含 `int n`）中，还未实现的部分是什么？**

   答：在第二个版本中，对于 `drawBezierCurve(vector<Point> &points, int n, color &c)` 函数的具体实现还未完成，可能是在计划中实现通过（m×3+1）控制点，绘制 `m` 段 `n-1` 次 Bezier 曲线。

7. **对于 `drawPolygonalLine` 函数中的绘制算法，是否了解绘制的原理？**

   答：是的，`drawPolygonalLine` 函数使用 OpenGL 的 `GL_LINE_STRIP` 模式，通过迭代遍历输入的点序列，绘制折线。

8. **如果要在 `drawPolygonalLine` 函数中添加绘制线段的粗细，应该如何修改代码？**

   答：在绘制前使用 `glLineWidth` 函数设置线段的粗细，可以通过增加一个新的参数传递粗细值，并在函数内调用 `glLineWidth`。

9. **`drawBezierCurve` 函数中的颜色设置和点大小设置，是否影响所有后续 OpenGL 操作的颜色和点大小？**

   答：是的，`glColor3f` 和 `glPointSize` 函数设置的颜色和点大小会影响之后所有使用的颜色和点大小，直到再次调用相应的函数进行修改。

10. **如果要在 `drawBezierCurve` 函数中添加绘制线段的粗细，应该如何修改代码？**

    答：在函数内添加 `glLineWidth` 函数调用，并在绘制前设置线段的粗细值。